

CPE477/ECG677 – Embedded Security & Machine Learning

Design Assignment 2

DO NOT REMOVE THIS PAGE DURING SUBMISSION:

Name: Chetan Dahal

Group-Partner Name:

Email: dahalc1@unlv.nevada.edu

Github Repository link (root): <https://github.com/autonomize009/autonomize009-Assign>

Youtube Playlist link (root):

<https://www.youtube.com/playlist?list=PLVIuzDVvAe9iHvuCgkgMcRMypPbO1yoZG>

Follow the submission guideline to be awarded points for this Assignment.

Submit the following for all Assignments:

1. In the document, for each task submit the modified or included code (from the base code) with highlights and justifications of the modifications. Also include the comments. If no base code is provided, submit the base code for the first task only.
2. Create a private Github repository with a random name (no CPE477/677, Lastname, Firstname). Place all assignments under the root folder, sub-folder named Assignmentn, with one document and one video link file for each lab, place modified c files named as main.c.
3. If multiple 'c' or 'h' files or other libraries are used, place these files inside the folder.
4. The folder should have a) Word document (see template), b) source code file(s) with other 'c' and 'h' include files, c) text file with youtube video links (see template).
5. Submit the pdf file in canvas before the due date. The root folder of the github assignment directory should have the documentation and the text file with youtube video links.
6. Organize your youtube videos as playlist under the name "EMBSEC&ML". The playlist should have the video sequence arranged as submission or due dates.
7. Only submit pdf documents. Do not forget to upload this document in the github repository and in the canvas submission portal.

1. Goal: Explain what is explored in this assignment and what was accomplished?

In this assignment, the integration of X-NUCLEO-IKS4A1 with STM32F401RE-NUCLEO Board was explored. It involved working with LSM6DSV16X and LIS2MDL sensors to obtain accelerometer, gyroscope and magnetometer reading. After completing this assignment, following things were accomplished:

- Learnt about calibration procedure of accelerometer, gyroscope and magnetometer.
 - Understood the way to correct the IMU sensor readings using calibration coefficients.
 - Developed machine learning model like linear regression, polynomial regression and decision tree using Python.
 - Explored whether the calibrated Euler's angles (roll, pitch and yaw) could be predicted from raw IMU sensor readings and raw Euler's angles using machine learning models.
 - Learnt about the metrics to compare the results of different machine learning models.
2. Screenshots of the IDE, physical setup, debugging process – Provide screenshot of successful compilation, screenshots of graphs, etc.

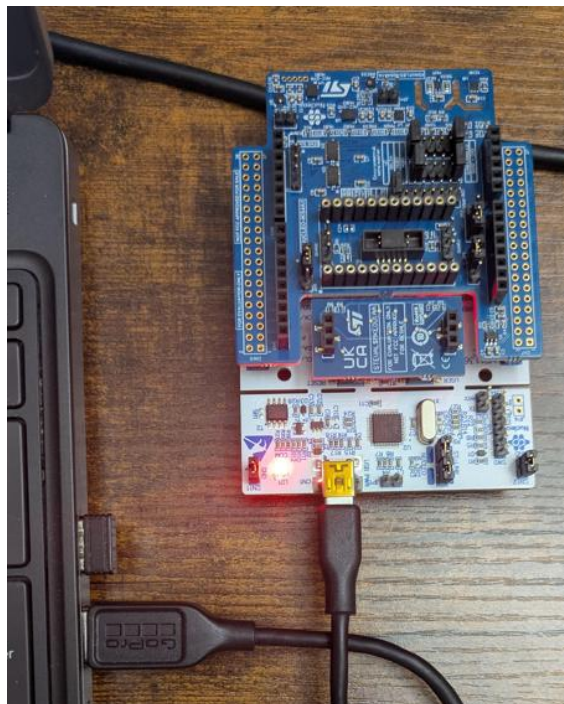


Figure 1: Physical setup of the STM32F401RE and X-NUCLEO-IKS4A1 board

Physical Setup

The X-NUCLEO-IKS4A1 sensor board was connected to the STM32F401RE MCU and the MCU was connected to the laptop via USB as shown in Figure 1.

Calibration Process

The “AccelerometerCalibration” code was loaded into STM32 MCU to start the accelerometer calibration process. MEMS Studio was used to collect the sensor data, which then uses MotionAC library under X-CUBE-MEMS1 software to calibrate the accelerometer. Dynamic calibration mode was used for calibrating the accelerometer, where the sensor board was gently rotated about different axes to obtain at least “OK” level of calibration. Thus obtained offset and scale factor were then recorded and noted down. Figure 2 shows the accelerometer calibration process performed. All the accelerometer offset were 0 and the scale factor were 1.

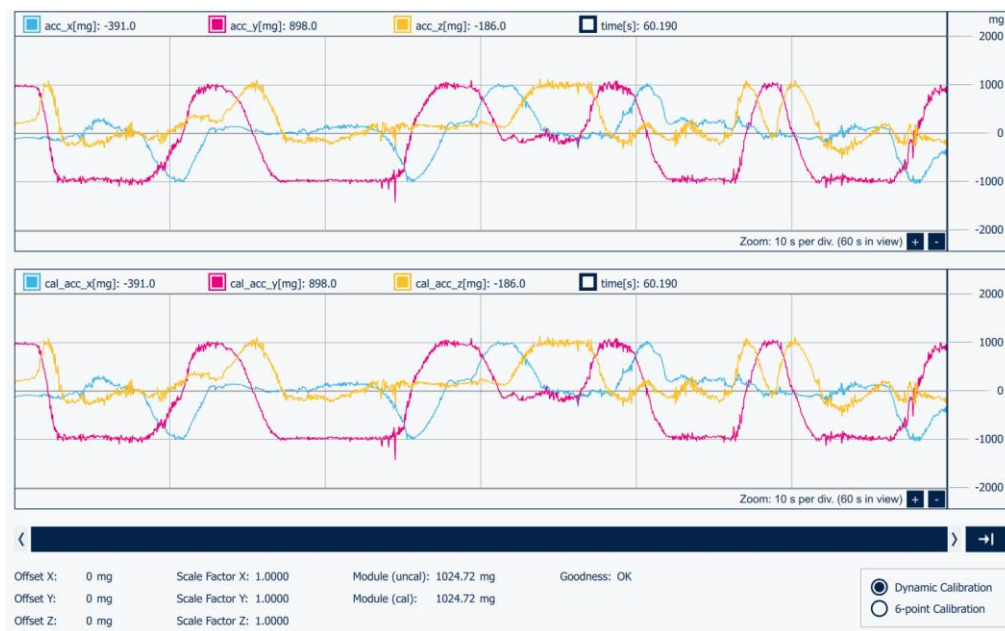


Figure 2: Accelerometer calibration process

Similar process was followed to obtain gyroscope reading for calibration process. “GyroscopeCalibration” application under X-CUBE-MEMS1 software was loaded into the MCU and readings of gyroscope were obtained in MEMS Studio. The sensor board was kept stationary on the table for gyroscope calibration for certain times until the calibration coefficient were

sufficiently stable. The stable bias of gyroscope in X, Y and Z direction (as seen in Figure 3) was then noted down for later correction.



Figure 3: Gyroscope calibration process

Finally, “MagnetometerCalibration” application under X-CUBE-MEMS1 was loaded into the MCU and MEMS Studio was used for obtaining the magnetometer readings. Magnetometer calibration process involved moving the sensor board in 8-pattern with simultaneous rotation of the board such that the motion of board covers entire spherical surface as shown in Figure 4.

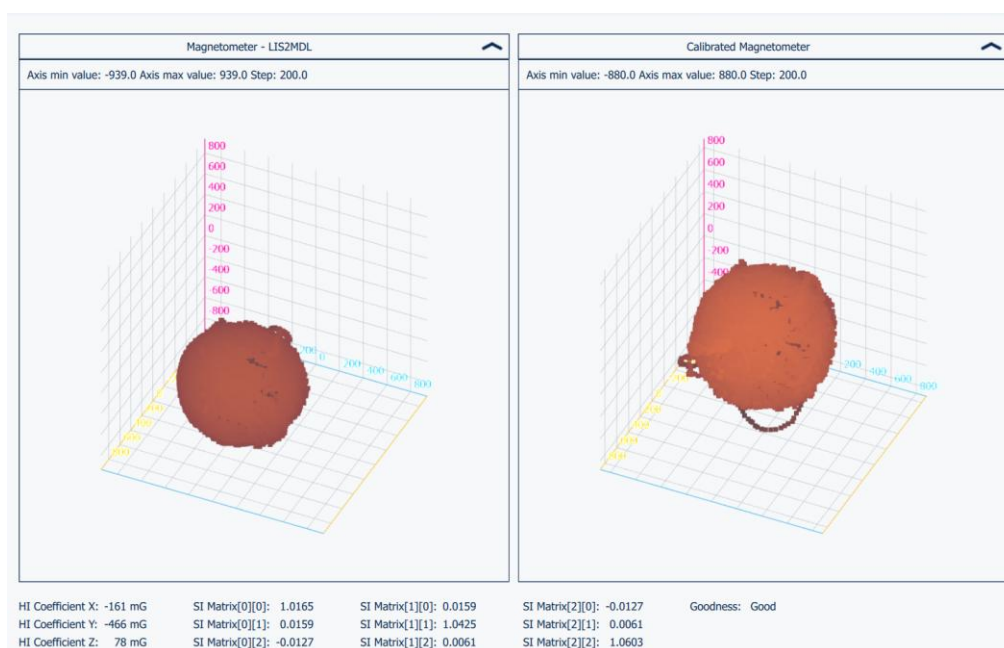


Figure 4: Magnetometer Calibration

After almost the entire spherical area was covered and the calibration state was good, the hard iron coefficient in X, Y, Z direction along with the soft iron matrix data were noted down.

Sensor Readings Correction

The raw IMU readings were corrected using calibration coefficients. The accelerometer offset recorded was in mg, gyroscope bias was in mdps and magnetometer HI-coefficient was in mG (uT = mG x 0.1). So, the following formula were used for the correction step:

$$Corrected_{acc} = (Raw_{acc} - acc_offset) * acc_scale_factor$$

$$Corrected_{gyro} = Raw_{gyro} - (gyro_bias/1000)$$

$$Corrected_{mag} = (Raw_{mag} - (HI_{coeff} * 0.1)) SI_{matrix}$$

Following function of python code applies the calibration coefficients for sensor reading correction.

```
# function to correct sensor data using calibration parameters
def apply_calib(d, acc_off, acc_s, gyr_b, mag_hi, mag_si):
    out = d.copy()
    A = out[["acc_x[g]", "acc_y[g]", "acc_z[g]"]].to_numpy(float)
    G = out[["gyro_x[dps]", "gyro_y[dps]", "gyro_z[dps]"]].to_numpy(float)
    M = out[["mag_x[uT]", "mag_y[uT]", "mag_z[uT]"]].to_numpy(float)

    out[["acc_x[g]", "acc_y[g]", "acc_z[g]"]] = (A - acc_off) * acc_s
    out[["gyro_x[dps]", "gyro_y[dps]", "gyro_z[dps]"]] = (G - (gyr_b)/1000.0)
    out[["mag_x[uT]", "mag_y[uT]", "mag_z[uT]"]] = (M - (mag_hi*0.1)) @ mag_si.T
    return out
```

Regression/Tree Approach

For obtaining the input feature, the sensor readings were loaded as the pandas dataframe and the raw accelerometer, gyroscope and magnetometer readings were extracted. These values were then calibrated using abovementioned process. Both raw and calibrated Euler angles were the calculated using these raw and calibrated IMU data.

First the accelerometer and magnetometer data were smoothed by using rolling average to remove the noisy data as seen in the following portion of code.

```
# function to perform rolling average on sensor data for smoothing the data
def rolling_mean(dfcols, w):
    if (w and w > 1):
        return dfcols.rolling(w, min_periods = 1, center = True). mean()
    else:
        return dfcols

# function to calculate Euler angles from given IMU sensor data
def compute_euler(df_sensor):
```

```
acc = rolling_mean(df_sensor[["acc_x[g]", "acc_y[g]", "acc_z[g]"]],
    ROLLING_WINDOW).to_numpy()
mag = rolling_mean(df_sensor[["mag_x[uT]", "mag_y[uT]", "mag_z[uT]"]],
    ROLLING_WINDOW).to_numpy()
```

Accelerometer readings were used to calculate the roll and pitch. After calculating roll and pitch, these values along with the magnetometer readings were used to obtain the tilt corrected yaw data as shown in following code portion.

```
# Roll/Pitch from accelerometer (deg)
roll = np.degrees(np.arctan2(ay, az))
pitch = np.degrees(np.arctan2(-ax, np.sqrt(ay**2 + az**2)))

# Tilt-compensated yaw from magnetometer (deg) using roll/pitch above
cr, sr = np.cos(np.radians(roll)), np.sin(np.radians(roll))
cp, sp = np.cos(np.radians(pitch)), np.sin(np.radians(pitch))
mx, my, mz = mag[:,0], mag[:,1], mag[:,2]
m_xc = mx*cp + my*sr*sp + mz*cr*sp
m_yc = my*cr - mz*sr
yaw = np.degrees(np.arctan2(m_yc, m_xc))
```

The raw yaw value suffers from wrapping problem as the angle is constrained between -180 to +180 degrees. So, there is problem of sudden jump in angles and hence raw yaw was unwrapped to produce continuous angle.

```
# unwrap raw yaw
yaw_raw_unwrapped = np.unwrap(np.deg2rad(yaw_raw))
yaw_raw_unwrapped_deg = np.rad2deg(yaw_raw_unwrapped)
Y_raw_df["yaw_deg"] = yaw_raw_unwrapped_deg
```

The training, validation and test dataset was then built in 60%/20%/20% ratio.

```
# building datasets (converting to numpy array)
Y_raw_all = Y_raw_df[targets].to_numpy(float) # raw Euler (train labels)
Y_cal_all = Y_cal_df[targets].to_numpy(float) # calibrated Euler (true labels)
X_raw = df_raw[input_features].to_numpy(float)

N = len(X_raw)
indices = np.arange(N)

# create train / val / test splits on indices
idx_tr, idx_tmp = train_test_split(indices, test_size = 0.4, random_state =
RANDOM_STATE, shuffle = True)
idx_val, idx_te = train_test_split(idx_tmp, test_size = 0.5, random_state =
RANDOM_STATE+1, shuffle = True)

# split train/validation/test dataset
X_tr = X_raw[idx_tr]
```

```

X_val = X_raw[idx_val]
X_te = X_raw[idx_te]

Y_tr_raw = Y_raw_all[idx_tr] # raw Euler labels for training
Y_val_raw = Y_raw_all[idx_val] # raw Euler labels for validation

```

Gridsearch was performed to tune hyperparameter for polynomial regression and decision training using the validation set. Polynomial regression for degree 2 and 3 were tried while different values of maximum depth and minimum samples leaf were tried for decision tree. The best model was then fit for these cases along with linear regression. Raw IMU readings as input features and raw Euler angles as true label were used for training each model.

Metrics (RMSE, MAE and R^2) were calculated for both training and test dataset. The predicted value of Euler angles was compared with calibrated Euler angle to observe whether the machine learning model can predict the calibrated Euler angles from raw sensor data. Following function was defined for metrics calculation.

```

# function for metrics calculation
def compute_metrics(name, Y_true, Y_pred, idx_subset):
    """
    Compute RMSE/MAE/R2 on a subset of indices.
    Roll & pitch: plain difference
    Yaw: wrapped difference in [-180, 180]
    """
    yt = Y_true[idx_subset] # (N_sub, 3)
    yp = Y_pred[idx_subset] # (N_sub, 3)

    # diff array same shape
    diff = np.zeros_like(yp)

    # roll, pitch: normal subtraction
    diff[:, 0] = yp[:, 0] - yt[:, 0]
    diff[:, 1] = yp[:, 1] - yt[:, 1]
    # yaw: wrapped difference
    diff[:, 2] = angle_diff_deg(yp[:, 2], yt[:, 2])

    # RMSE / MAE from diff
    rmse = np.sqrt(np.mean(diff**2, axis=0))
    mae = np.mean(np.abs(diff), axis=0)

    # For R2, build "aligned" predictions so residuals match diff for yaw
    yaw_pred_aligned = yt[:, 2] + diff[:, 2]
    Y_true_all = yt
    Y_pred_all = np.column_stack([yp[:, 0], yp[:, 1], yaw_pred_aligned])

    r2 = r2_score(Y_true_all, Y_pred_all, multioutput='raw_values')

```

The obtained results were then displayed and plotted.

Results

Figure 5 shows the result of machine learning prediction. First, the hyperparameter tuning selected degree 3 polynomial as the suitable one for polynomial regression. Max depth of 15 and minimum leaf sample of 1 was selected during tuning. Then, after training, the metrics were calculated for both training + validation dataset and test dataset.

```
Best Polynomial degree: 3
Best Tree params: {'max_depth': 15, 'min_samples_leaf': 1}

Linear regression (Train+Val) metrics vs Calibrated Euler
RMSE (roll,pitch,yaw): [12.756  3.49  31.849]
MAE (roll,pitch,yaw): [ 7.491  2.394 23.357]
R2 (roll,pitch,yaw): [0.91  0.978 0.265]

Polynomial Regression (Train+Val) metrics vs Calibrated Euler
RMSE (roll,pitch,yaw): [ 2.065  0.88 30.184]
MAE (roll,pitch,yaw): [ 1.279  0.559 22.382]
R2 (roll,pitch,yaw): [0.998 0.999 0.34 ]

Decision Tree (Train+Val) metrics vs Calibrated Euler
RMSE (roll,pitch,yaw): [ 0.639  0.475 30.451]
MAE (roll,pitch,yaw): [ 0.302  0.234 22.417]
R2 (roll,pitch,yaw): [1. 1. 0.328]

Linear regression (Test) metrics vs Calibrated Euler
RMSE (roll,pitch,yaw): [13.078  3.475 32.031]
MAE (roll,pitch,yaw): [ 7.627  2.387 23.43 ]
R2 (roll,pitch,yaw): [0.91  0.977 0.28 ]

Polynomial regression (Test) metrics vs Calibrated Euler
RMSE (roll,pitch,yaw): [ 1.957  0.957 30.073]
MAE (roll,pitch,yaw): [ 1.288  0.592 22.275]
R2 (roll,pitch,yaw): [0.998 0.998 0.366]

Decision Tree (Test) metrics vs Calibrated Euler
RMSE (roll,pitch,yaw): [ 3.181  1.809 30.219]
MAE (roll,pitch,yaw): [ 1.268  0.935 22.377]
R2 (roll,pitch,yaw): [0.995 0.994 0.36 ]
```

Figure 5: Performance Metrics

As seen in Figure 5, all 3 models performed well for roll and pitch prediction. For the linear regression test dataset, the RMSE value for the roll and pitch were 13.078 and 3.475 with R^2 value as 0.91 and 0.977. Similarly, for polynomial regression, RMSE value for roll and pitch are 1.957 and 0.957 respectively and for decision tree the values are 3.181 and 1.809 respectively. Based on the RMSE value, polynomial regression with degree of 3 is the best model for predicting roll and pitch using raw IMU readings. The R^2 value for the prediction is also high for polynomial regression leading to proper explanation of variance.

However, for yaw, all three models provided terrible prediction. The RMSE value for yaw prediction seen in the test dataset were 32.031, 30.073 and 30.219 for linear regression, polynomial regression and decision tree respectively. Although the error was high, among them polynomial regression showed slightly better result. The R^2 value were also poor for all three models for yaw prediction indicating the poor prediction results.

Hence, polynomial regression can be used in general for predicting the calibrated Euler angle values directly from raw IMU sensor readings, keeping in mind that yaw value might not be a good prediction.

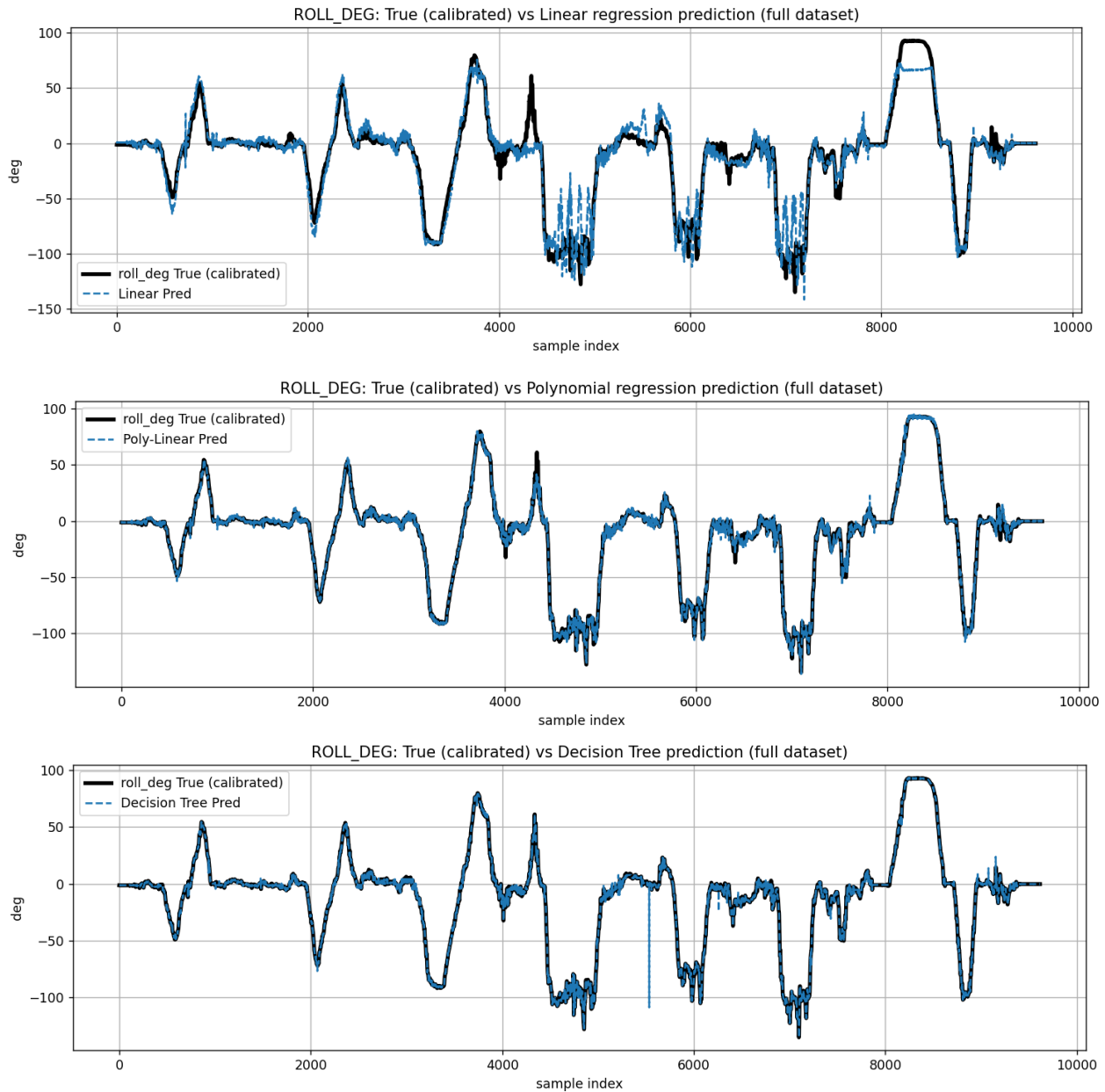


Figure 6: Comparison graph between calibrated roll and predicted roll for all three models

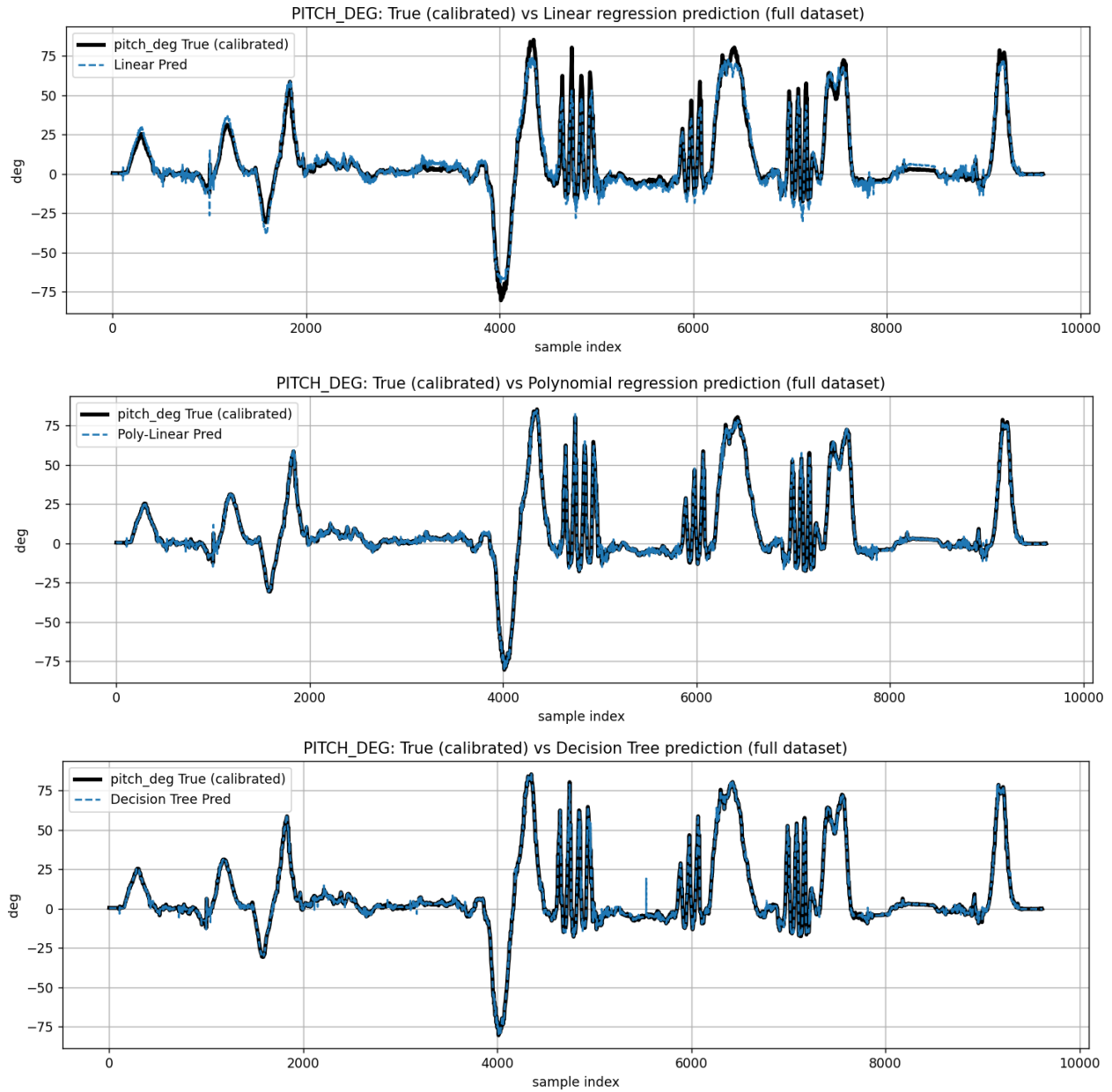


Figure 7: Comparison graph between calibrated pitch and predicted pitch for all three models

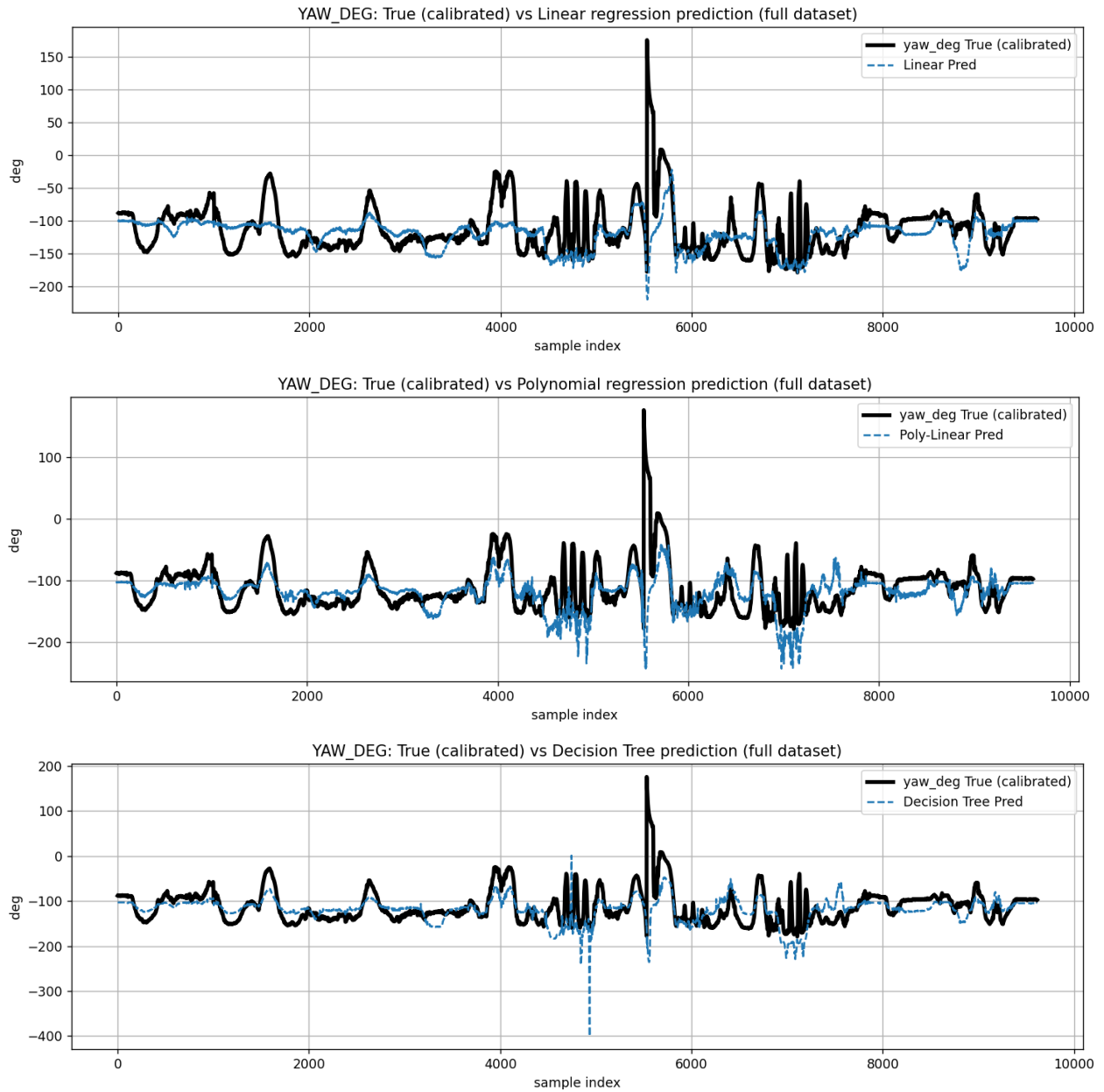


Figure 8: Comparison graph between calibrated yaw and predicted yaw for all three models

Figure 6,7,8 illustrates the result obtained from the machine learning models. As seen, the roll and pitch values were predicted well by all 3 models with polynomial regression being the best one, while for the yaw value, all three models showed poor performance, indicating that it may not be the good idea to predict calibrated yaw value from the raw IMU sensor data.

3. Declaration

I understand the Student Academic Misconduct Policy -

<http://studentconduct.unlv.edu/misconduct/policy.html>

“This assignment submission is my own, original work”.

Chetan Dahal